

Advanced Engineering Notes: Haptic Systems, Voltage-Mode Dynamics, and Register-Level Controls

A COMPREHENSIVE ACADEMIC & PRACTICAL BENCH GUIDE

Section 1: Foundations of Haptic Systems & Control Loop Stability

Haptic interfaces act as bidirectional physical ports linking a human operator with a virtual or remote environment. Unlike traditional visual or auditory feedback channels, a haptic feedback device forms a tightly coupled, bilateral closed-loop system where physical energy is explicitly exchanged between the user and the actuator. This exchange mandates the categorization of haptic displays into two fundamental topologies:

- **Impedance Displays:** These devices read the instantaneous user position (or velocity) via high-resolution position sensors and compute an appropriate reaction force to command to the motor. Impedance displays require highly backdrivable, low-inertia mechanisms so that unconstrained motion feels completely unhindered.
- **Admittance Displays:** These devices employ a force sensor to measure the active input force exerted by the user and command a corresponding velocity or position shift via a highly geared, non-backdrivable robotic assembly.

The core challenge in impedance-type haptic rendering is the preservation of **passivity**. In an ideal continuous physical system, virtual elements like springs can only store and release energy, while virtual dampers dissipate it. However, when implemented over digital microcontrollers, haptic systems suffer from spatial quantization (encoder resolution limits) and discrete sampling delays. This discrete lag introduces an artificial phase shift, turning passive virtual environments into net energy generators. When a user strikes a rigid virtual boundary (a virtual wall), this phantom energy manifests as high-frequency vibrations, jitter, or unstable oscillatory bursts that can cause severe safety hazards on a test bench.

Section 2: The Core Paradigm Shift — Voltage-Mode vs. Current-Mode Control

Traditional robotics textbooks dictate that electromagnetic actuators must be driven via **Current-Mode Control** (Force/Torque Control). Because motor current (i) maps directly and linearly to output torque ($\tau = K_t \cdot i$), managing current directly yields precise, linear command over the rendered force field. Under this dogma, any voltage variations or back-electromotive force disturbances are actively tracked and neutralized by an ultra-fast inner current loop.

However, the seminal research paper *"Haptic Performance using Voltage-Mode Motor Control"* by Jorge Juan Gil and Iñaki Díaz (IEEE Transactions on Industrial Electronics, 2019) exposes a profound vulnerability in this paradigm: **current-mode control strips away a vital structural asset for system stability**. By eliminating back-EMF variations, a current-controlled driver removes the natural electrical damping inherent to the motor windings. Conversely, **Voltage-Mode Control** forces an implicit, analog, lag-free physical viscosity (damping)

into the actuator, rendering it structurally superior for simulating Exceptionally stiff, stable virtual surfaces without triggering high-frequency jitters.

Section 3: The Mechanical-Electrical Dissection & Back-EMF Damping

When a motor shaft is forced to rotate by a user's hand, the armature coils cut through permanent magnetic flux lines, transforming the actuator into a generator. This process induces an internal back-electromotive force voltage (e) directly inside the copper windings at the speed of light:

$$e = K_e \cdot \omega$$

Where K_e is the motor's back-EMF constant and ω is the instantaneous angular velocity of the shaft. By considering the standard steady-state DC motor voltage drop equation across internal coil resistance (R) and ignoring small winding inductances:

$$V = R \cdot i + e \Rightarrow i = (V - e) / R$$

Substituting this current expression into the core torque equation ($\tau = K_t \cdot i$) reveals the true governing torque expression under an applied terminal voltage (V):

$$\tau = (K_t / R) \cdot V - (K_t \cdot K_e / R) \cdot \omega$$

This derivation exposes a hidden passive damping term, structurally defined as the ****Electromechanical Damping Coefficient (b_e)****:

$$b_e = (K_t \cdot K_e) / R$$

This internal viscosity acts as a **Free Analog Sponge**. Unlike software-calculated damping, which relies on noisy encoder differentiation and suffers from digital processing delay, b_e features zero sampling latency and zero quantization noise. It serves as a continuous, physical energy sink that instantly absorbs impact energy at the virtual wall interface, stabilizing the coupled hand-device interaction naturally.

Section 4: The Gearbox Multiplier Matrix (N^2 Scale Dynamics)

When a haptic mechanism incorporates a mechanical transmission (such as a capstan drive, belt reduction, or planetary gearbox) with a strict reduction ratio of N , the mechanical and electrical properties felt at the user interaction handle scale unevenly according to a torsional reflection matrix:

- **Reflected Inertia:** The mechanical mass (inertia) of the motor rotor felt at the handle scales up by the square of the ratio: $N^2 \cdot J_m$. This is generally undesirable, as it adds heavy parasitic mass to free-space trajectories, diminishing transparency.
- **Reflected Electromechanical Damping:** Crucially, the internal back-EMF viscosity felt at the handle also scales up by the square of the ratio: $N^2 \cdot b_e$.

This disparity forms a powerful engineering compromise. Designers can deploy a tiny, ultra-lightweight, low-inertia motor to keep free-space unconstrained motion crisp and nimble, while relying on a low-friction

mechanical transmission to multiply the internal "magnetic honey effect" by `N2`. This delivers immense stabilizing damping right at the virtual wall boundary without adding physical weight or complex software filters.

Section 5: Critical Trade-Off Triangle (Voltage vs. Current)

Choosing between these two actuation paradigms requires evaluating three distinct performance boundaries:

Performance Parameter	Voltage-Mode Actuation	Current-Mode Actuation
Stability & Stiffness Limits	Achieves an extraordinarily high Critical Stiffness (K_{crit}) limit before breaking into jitter.	Highly prone to boundary instability; requires artificial software damping to remain stable.
Free-Space Backdrivability	Feels slightly viscous during free motion because the organic back-EMF drag is always active.	Wins cleanly; the current loop actively tracks and nullifies back-EMF, driving internal drag to zero.
Thermal Invariance	Suffers from thermal drift. As coils heat up and resistance R increases, b_e drops, softening the wall.	Maintains perfect consistency; automatically increases voltage to sustain the target current regardless of temp.

EXPLAINING BENCH FAILURES & ACTIVE COMPENSATION RULES

The Driver Transition Trap: Swapping an uncompensated voltage driver (like the legacy L298N) for a high-efficiency driver operating as a tight current amplifier (such as a Cytron MD10C in certain loop setups) deletes the native back-EMF damping cushion. Without software-simulated damping to compensate, energy accumulates rapidly at the boundary edge, causing the control loop to vibrate aggressively and break couplers.

The Active Remedy: When utilizing an active current-sensing driver (e.g., LMD18200T), *Active Back-EMF Compensation* must be coded manually. Designers must calculate clean velocity estimates via filtering and subtract an explicit damping vector to artificially rebuild the stabilizing cushion destroyed by current control.

Section 6: Register-Level Optimization — Direct Port Manipulation

1. The Architectural Bottleneck

Standard Arduino core functions like `digitalWrite()` and `digitalRead()` introduce immense computational overhead, requiring 4 to 5 microseconds per execution line. They spend critical CPU cycles executing lookups, parsing pin-to-timer maps, validation checks, and masking interrupts. In high-frequency haptic control loops running at 1 kHz or higher, this latency injects severe phase lag. By implementing **Direct Port Manipulation**, we bypass this abstraction layer entirely, writing directly to the microcontroller's I/O registers in nanoseconds (< 20 ns, or a single CPU clock cycle).

2. Renesas RA4M1 (Arduino Uno R4) 32-Bit Registry Architecture

Unlike historic 8-bit AVR microcontrollers, the Renesas RA4M1 relies on a sophisticated 32-bit Arm Cortex-M4 memory-mapped layout governed by the Board Support Package (BSP) structures:

- **PDR (Port Direction Register):** Governs the input/output state of the pin. Writing a `1` defines the pin as an output; a `0` defines it as an input.

```
R_PORT1->PDR |= (1 << 11); // Set Port 1, Pin 11 to Output
```

- **PODR (Port Output Data Register):** Directs the physical electrical state of pins configured as outputs. Writing a `1` drives the rail HIGH, while a `0` forces it LOW.

```
R_PORT1->PODR |= (1 << 11); // Force Pin High Instantly
```

- **PIDR (Port Input Data Register):** A read-only register that snapshots the raw electrical state of all input pins across the port block in a single cycle.

```
uint32_t port3State = R_PORT3->PIDR; // Absolute snapshot of Port 3
```

Section 7: Software Implementation & Fast Register Scripts

The following scripts implement direct bitwise masking to achieve ultra-low latency reading and driving states on the Uno R4 Minima:

```
// =====  
// HARDWARE BITMASK INITIALIZATION (Place within void setup())  
// =====  
  
// 1. Configure Pin 4 (Port 1, Pin 11) as an Output for Motor Direction Control  
R_PORT1->PDR |= (1 << 11);  
  
// 2. Configure Pin 2 (Port 3, Pin 1) and Pin 3 (Port 3, Pin 2) as Inputs for Encoder  
R_PORT3->PDR &= ~((1 << 1) | (1 << 2));
```

```
// =====  
// ULTRA-FAST QUADRATURE ENCODER REGISTER READER  
// =====  
volatile long encoderPos = 0;  
volatile int lastEncoded = 0;  
  
void updateEncoderRegisterMode() {  
    // Capture Port 3 register state in one single CPU cycle  
    uint32_t port3State = R_PORT3->PIDR;  
  
    // Isolate bit 1 (Pin 2) and bit 2 (Pin 3) using shifting and bitwise AND masking  
    int MSB = (port3State >> 1) & 0x01;  
    int LSB = (port3State >> 2) & 0x01;  
  
    int encoded = (MSB << 1) | LSB;  
    int sum = (lastEncoded << 2) | encoded;  
  
    if(sum == 0b0010 || sum == 0b1011 || sum == 0b1101 || sum == 0b0100) encoderPos++;  
    if(sum == 0b0001 || sum == 0b0111 || sum == 0b1110 || sum == 0b1000) encoderPos--;  
  
    lastEncoded = encoded;  
}
```

```
// =====
// LOW-LATENCY DIRECTION ACTUATION EXECUTION
// =====
void executeDirectionActuation(bool directionState) {
    if (directionState) {
        R_PORT1->PODR |= (1 << 11); // Direct register write to Pin 4 HIGH
    } else {
        R_PORT1->PODR &= ~(1 << 11); // Direct register write to Pin 4 LOW
    }
}
```

Section 8: Physical Hardware Connection Blueprints & Workspace Calibration

CRITICAL HARDWARE PIN WARNING

Because the underlying code communicates directly with hardwired 32-bit hardware register spaces, components must be mapped exactly to the physical pins designated below. Bypassing these specific pins completely breaks the logic.

Driver Option A Blueprint: Cytron MD10C R3 (Sign-Magnitude Logic)

- Arduino Uno R4 Pin 5 (Port 1, Pin 2) → **Cytron PWM Input** (Carries high-speed 20 kHz pulses)
- Arduino Uno R4 Pin 4 (Port 1, Pin 11) → **Cytron DIR Input** (Drives direct register direction changes)
- Arduino Uno R4 GND Pin → **Cytron GND Terminal** (Establishes uniform logic reference)
- POWER + / - → Tied directly to the respective positive and negative terminals of the 12V DC power supply
- MOTOR A / B → Tied directly to Motor Armature Terminal 1 and Terminal 2

Driver Option B Blueprint: L298N H-Bridge (Dual-Input Direction Logic)

- Arduino Uno R4 Pin 5 (Port 1, Pin 2) → **L298N ENA Pin** (Remove local jumper block; carries 10 kHz pulses)
- Arduino Uno R4 Pin 4 (Port 1, Pin 11) → **L298N IN1 Pin** (Controls current vector Forward)
- Arduino Uno R4 Pin 7 (Port 1, Pin 3) → **L298N IN2 Pin** (Controls current vector Reverse)
- Arduino Uno R4 GND Pin → **L298N GND Pin** (Crucial: Tie Arduino ground and 12V supply ground together here)
- VCC (12V Terminal) → Connected to the positive (+) terminal of the 12V power supply

Shared Quadrature Encoder Interface Mapping

- Encoder VCC / GND → Connected to the Arduino Uno R4 5V and GND power rails
- Encoder Channel A (CLK) → **Pin 2** (Port 3, Pin 1 / Memory Register Bit: P301)
- Encoder Channel B (DT) → **Pin 3** (Port 3, Pin 2 / Memory Register Bit: P302)

3. Workspace Calibration Protocol

If the entire setup is fully wired and powered, but moving the haptic handle past the 400-count boundary causes the motor to pull your hand deeper into the wall (instigating a destructive **Positive Feedback Loop**) instead of pushing back outward, do not alter your physical wire layout. Navigate to the top of your configuration script and toggle the boolean state flag:

```
const bool INVERT_MOTOR_DIRECTION = true; // Toggle to false if positive feedback occurs
```

This flag operation instantly realigns the microcontroller's mathematical force command matrix with your physical wiring topology without modifying your hardware bench space.

Section 9: Core Glossary & Operational Terminology

H-Bridge Matrix — A solid-state switching network containing four discrete switching elements structured to govern current vector directions across an active inductive load, managing rotational spin directions.

Bootstrap Gate Drive — A capacitive pumping layout designed to capture low-side electrical charge to step up gate-drive potentials past the main supply voltage (V_S), forcing high-side N-channel DMOS elements into full saturation.

Stiction (Static Friction) — The mechanical breakaway friction threshold that a stationary shaft assembly must overcome before transitioning into continuous kinetic rotation.

Motor Stall Current — The peak electrical current consumed when an electric motor's shaft is physically restricted from rotating while full voltage is sustained. Under this condition, the windings act purely as a low-value static resistor.

Integral Windup / Sticky Integral — A control loop anomaly where a persistent error accumulates an excessively large integral term inside a PI/PID loop. Upon releasing the constraint, this historical error memory keeps the loop saturated, causing virtual haptic boundaries to feel sticky or glued.

State Resetting — A haptic control paradigm where historical tracking variables (like `integralError`) are instantaneously wiped to zero the moment a physical boundary condition changes, eliminating transition lag.